통합 계좌 관리 시스템 기술 명세서 (Core Technologies)

1. 보안 및 데이터 보호 (Security & Data Protection)

1.1 바인드 변수를 활용한 SQL 인젝션 방지

사용자 입력값을 SQL 문장에 직접 연결하지 않고, 바인드 변수(`:1`, `:2`)를 사용하여 데이터베이스 서버에 전달합니다. 이는 악의적인 SQL 주입 공격을 원천 차단하며, 쿼리 실행 계획을 재사용할 수 있게 하여 시스템 성능을 향상시킵니다.

핵심 코드:

사용자 ID를 안전하게 바인딩하여 조회

```
# 1. 현재 계좌 상태 및 기존 별칭 조회(본인 계좌인지 확인)
sql_select = "select alias from Account where account_num = :1 and customer_id = :2"

# 2. 별칭 업데이트
sql_update = "update Account set alias = :1 where account_num = :2"

# 3. 변경 이력 기록
sql_history = """
    insert into AccountHistory (account_num, action_type, before_value, after_value)
    values (:1, '별칭변경', :2, :3)
"""

conn = self.get_connection()
try:
    cursor = conn.cursor()

    # 계좌 소유주 및 현재 별칭 확인
    cursor.execute(sql_select, (account_num, customer_id))
```

1.2 비밀번호 마스킹 처리 (getpass)

비밀번호 입력 시 화면에 글자가 노출되지 않도록 `getpass` 모듈을 사용합니다. 이는 오프라인 환경에서의 어깨너머 해킹(Shoulder Surfing)을 방지하는 필수 보안 조치입니다.

핵심 코드:

```python
# 아이디 중복 검증
if self.db.get_customer(user_id):
    print("[!] 이미 존재하는 아이디입니다. 다른 아이디를 사용해주세요.")
    return

password = getpass.getpass("비밀번호: ").strip()
```

1.3 트랜잭션 원자성 보장 (Rollback Strategy)

금융 거래 중 오류가 발생할 경우 `conn.rollback()`을 호출하여 해당 작업 세션의 모든 변경 사항을 취소합니다. "전부 처리되거나, 아예 처리되지 않아야 한다"는 원자성(Atomicity)을 보장하여 자금 증발 사고를 예방합니다.

핵심 코드:

```python
def execute_transfer(self, from_account, to_account, amount):
        # 1. 출금 처리
        cursor.execute(sql_update_bal, (-amount, from_account))
        cursor.execute(sql_insert_tx, (
            from_account,       # :1
            'WITHDRAWAL',       # :2
            amount,             # :3
            new_from_bal,       # :4
            f"{to_account}에게 이체" # :5
        ))
        # 2. 입금 처리
        cursor.execute(sql_update_bal, (amount, to_account))
        cursor.execute(sql_insert_tx, (
            to_account,         # :1
            'DEPOSIT',          # :2
            amount,             # :3
            new_to_bal,         # :4 (이게 빠져있었습니다!)
            f"{from_account}로부터 입금" # :5
        ))
        # 출금 계좌 로그
        self.log_history(cursor, from_account, 'TRANSFER_OUT', current_balance, new_from_bal)
        # 입금 계좌 로그
        self.log_history(cursor, to_account, 'TRANSFER_IN', to_balance, new_to_bal)
        # 모든 작업이 정상적으로 끝났을 때만 DB에 반영
        conn.commit()
        return True, "계좌 이체가 성공적으로 완료되었습니다."

    except oracledb.DatabaseError as e:
        # 중간에 오류가 발생하면 무조건 롤백하여 자금 증발 방지
        conn.rollback()
        return False, f"시스템 오류로 모든 작업이 취소되었습니다. 사유: {e}"
```

2. 데이터 처리 및 객체 지향 설계 (Data Handling & OOP)

2.1 Fetch 메서드를 활용한 메모리 최적화

데이터의 성격에 따라 적절한 추출 메서드를 사용하여 메모리 사용을 최적화합니다.

- **`fetchone()`**: 로그인 검증, 단일 잔액 조회 등 1개의 결과만 필요한 경우.

```
# 계좌 소유주 및 현재 별칭 확인
cursor.execute(sql_select, (account_num, customer_id))
row = cursor.fetchone()
```

- **`fetchall()`**: 계좌 목록 조회, 검색 결과 출력 등 다수의 결과가 필요한 경우.

```
columns = [col[0] for col in cursor.description]
rows = cursor.fetchall()
```

2.2 Self 매개변수를 통한 상태 관리

클래스 내부 함수에서 `self`를 사용하여 데이터베이스 연결 객체(`conn`)나 로그인한 사용자 정보(`current_user`)를 객체 전체에서 공유하고 관리합니다.

```
class DatabaseManager:
    def __init__(self):

        self.username = 'C##BANK_ADMIN'
        self.password = '1234'

        self.dsn = oracledb.makedsn('192.168.219.102', 1521, service_name='FREE')

    def get_connection(self):
        """DB 커넥션 객체를 반환합니다."""
        try:
            return oracledb.connect(user=self.username, password=self.password, dsn=self.dsn)
        except oracledb.DatabaseError as e:
            error, = e.args
            print(f"[DB 연결 에러] 코드: {error.code}, 메시지: {error.message}")
            raise
```

3. 핵심 구현 기능 (Key Functional Modules)

3.1 시퀀스를 활용한 계좌번호 자동 발급

Oracle 데이터베이스의 `SEQUENCE` 객체를 활용하여 고유한 계좌번호를 생성합니다. 중복되지 않는 일련번호를 보장합니다.

핵심 **SQL:**

```sql
-- 계좌번호 뒷자리에 쓸 시퀀스 생성
CREATE SEQUENCE seq_acc_no START WITH 1000 INCREMENT BY 1;
```

3.2 자동이체 배치 처리 (Auto Transfer Batch)

사용자가 설정한 주기적인 이체 정보를 `AutoTransfer` 테이블에 관리하며, 시스템은 지정된 날짜(`transfer_day`)를 확인하여 대량의 이체 작업을 자동으로 처리합니다.

```sql
INSERT INTO AutoTransfer (from_account, to_bank_code, to_account, amount, transfer_day, is_active)
VALUES ('002-1000', '002', '003-1001', 5000, 16, 1);
```

```python
def process_auto_transfer_batch(self):
    """오늘자 자동이체 전체 실행 및 결과 요약 반환"""
    pending_list = self.get_pending_auto_transfers()
    if not pending_list:
        return False, "오늘 처리할 자동이체 내역이 없습니다."

    success_count = 0
    fail_count = 0
    results = []

    for task in pending_list:
        # 기존에 만들어둔 execute_transfer 함수 재사용
        # (로그 기록 기능이 포함된 버전을 호출합니다)
        success, message = self.execute_transfer(
            task['from_account'],
            task['to_account'],
            task['amount']
        )
```

3.3 계좌 이력 관리 (Account History/Log)

단순한 현재 잔액뿐만 아니라, 모든 금융 행위의 전/후 상태를 기록하여 투명성을 확보합니다.

핵심 코드:

```
def log_history(self, cursor, acc_num, action, before, after):
    """AccountHistory 테이블에 로그를 기록하는 공통 함수"""
    sql = """
        insert into AccountHistory (account_num, action_type, before_value, after_value)
        values (:1, :2, :3, :4)
    """
    try:
        # before와 after 값은 숫자일 수 있으므로 str()로 감싸서 문자열로 저장합니다.
        cursor.execute(sql, (acc_num, action, str(before), str(after)))
    except Exception as e:
        # 로그 기록 실패가 전체 이체 실패로 이어지지 않게 하려면
        # 여기서 예외처리를 하거나 프린트만 찍어줍니다.
        print(f"[경고] 로그 기록 실패: {e}")
```

---

4. 예외 처리 전략 (Error Handling)

시스템의 안정성을 위해 오류의 성격에 따라 대응 전략을 분리합니다.

예외 타입	대상 및 사유	대응 방식
IntegrityError	중복 가입, 제약조건 위반 (사용자 과실)	명확한 경고 메시지 출력 및 재입력 유도
DatabaseError	네트워크 단절, 서버 다운 (시스템 결함)	즉시 롤백 처리 및 시스템 관리자 알림

```
except oracledb.IntegrityError:
    conn.rollback()
    return False, "이미 등록된 데이터와 충돌이 발생했습니다."
except oracledb.DatabaseError as e:
    conn.rollback()
    return False, f"DB 오류 발생: {e}"
```